The CS Curriculum in the Age of Coding Agents

More questions than answers

Bill Pugh, University of Maryland

Professor Emeritus of Computer Science

pugh@umd.edu

Slides, PDF, and references: billpugh.github.io/talks/2026-10-12-UCSD

billpugh.github.io/talks/2026-10-12-UCSD

Questions, some answers and some guesses

- How is software development changing, and which changes will last?
- What skills should CS students have when they graduate?
- What do we know about how AI affects learning?
- What advice can we give current CS students?
- How should the CS curriculum change, from the first programming course to the capstone?
- How should we change our intro CS courses?

How much change should we prepare for?

¯_(ツ)_/¯

Very few knowledgeable people try to make detailed predictions further than two years into the future.

I'm going to pick a rate of change that I think is just a little bit conservative:

- Should be good preparation for all but the rapid-change scenarios
- If things change very rapidly, we will have moved in the right direction and can adjust

How is software development changing, and which changes will last?



billpugh.github.io/talks/2026-10-12-UCSD

Source code will be *typed* by AI

- "Typed," not "written." I mean the text of source files. Writing software is more than that.
- **Anthropic**, June 2026: more than 80% of code merged into their codebase was authored by Claude
- **Google**, April 2026: 75% of new code AI-generated and approved by engineers (25% in Oct 2024, 50% fall 2025)
- **Thorsten Ball**, co-creator of the Amp coding agent and author of *Writing a Compiler in Go*, July 2026: "Nobody in our team really writes code by hand anymore. I certainly don't." His team poll: 99, 95, 90 percent

Two companies and one engineer who used to write compilers by hand. One direction.

This becomes near-universal even if the models stop improving

Why not yet:

1. **Adoption lag**
2. **Harness headroom.** The agent scaffolding can improve independently of the model, some of it company-specific
3. **QA infrastructure.** Rerun every affected test, static analysis, style guides, checks that the code is not getting harder to maintain, before code is ready for review or use

None trivial. Most companies get there in about two years.

Prefer frameworks that are secure by design

- For example, prefer Rust to memory-unsafe languages
- The OWASP Secure by Design Framework
- Use abstractions that prevent entire categories of bugs
- Cedar (Amazon, 2023): an open-source language and engine for access control. Permissions are policies, separate from application logic, and the engine is formally modeled and its correctness properties proved with automated reasoning. cedarpolicy.com

Such frameworks make important properties and abstractions explicit in the code, making it easier for humans and agents to have a good conversation about them, and remove the concern that agents might not correctly implement them.

Offensive AI-powered cyberattacks demand better code

- The way even top development teams wrote software two years ago is no longer sufficient
- Using AI coding models to improve security is essential, but may not be sufficient
- Companies such as Google and Amazon are trying hard to move from C/C++ to Rust, even rewriting existing C/C++ code bases

What happens to code review

Boris Cherny, creator of Claude Code, Sept 2026, in response to "What to do about slop?": production code written by Claude "should have a higher bar than if it was written by a human." The bar: lint, tests, Claude-driven end-to-end tests, daily fuzzers, automated code and security review, automated refactoring.

> "Your job is to hold the bar on code quality."

He never says "read the code."

Simon Willison, May 2026: "I'm not reviewing every line of code that they write anymore, even for my production level stuff."

Agents get very good at anything with a clear pass/fail test

- Anything with a clear pass/fail test can be used for **reinforcement learning with verifiable rewards.** The models get exceptionally good at exactly that
- And an agent can iterate in a loop until every test passes

So: for a well-specified problem with an adequate validation harness, functional correctness is mostly no longer the worry.

The remaining issues are all the **other** things we expect of software.

What code review was always about

- Three studies over fifteen years: about **75% of review findings are maintainability**, 25% functional (Mäntylä & Lassenius 2009; Bacchelli & Bird 2013; Beller et al. 2014). Microsoft's own title: "Code reviews do not find bugs" (Czerwonka, Greiler, Tilford, 2015)
- Much of that 75% was not style. "When an experienced engineer reads a diff and says 'I don't understand this,' their confusion is the finding" (Allspaw, 2026). Comprehensibility is a property of a person meeting the code, not of the code alone
- Google's stated reason for code review: "code must act as a teacher for future developers." Readability, "maintaining norms": *that isn't how we do it here*
- Google tried **117 code and log metrics** to predict where its engineers said working code was slowing them down. **None worked.** Their explanation: that is a judgment comparing the code as it is to the code as it should be, and no metric can see the "should" (Jaspan & Green, 2023)

Three kinds of debt (Storey, 2026)

- **Technical debt** makes systems harder to change
- **Cognitive debt** makes systems harder to understand
- **Intent debt** makes it hard to know what the system is for

> "Generative AI does not remove the challenges of software engineering; it redistributes them."

Her own students: a senior team using AI to ship fast hit a wall in week eight, because "no one on the team could explain why certain design decisions had been made."

What injury might we incur by just using AI code that passes every test?

Suppose the tests are adequate and the code passes them. What might we lose?

- **The code may still be slop:** duplication, not the way this codebase does it, structure that resists the next change, a design nobody would have chosen
- When humans built the code, building it had benefits beyond the code:
  - the people who wrote it **understood** it, and could change it safely
  - the **reasons** for its shape were in someone's head, and often in the names and structure
  - review taught newcomers **how we do things here**, and the conversation produced "a shared understanding that neither party had prior to the discussion"

Accept generated code and you get the artifact without the side effects. The tests cannot see what is missing.

Code review will transform to change review

Marc Brooker, Distinguished Engineer, AWS, Sept 2026: "I believe that, long-term, humans have no role in routinely reviewing code."

Two different things are likely to happen:

1. **Many changes will not need any human review.** Meta's RADAR (2026): of 535K eligible low-risk, mostly bot-generated diffs, a risk model plus an LLM reviewer landed 62% (332K) with no human review and sent the other 38% to people. Everything outside the eligible categories still goes to human review
2. **Changes that do need review will be reviewed above the source code.** Meta's AfterVibe (2026) distills an agent session and its diff into a spec of intent, design decisions, and behavioral requirements, about one sixth the diff's size: "specifications, not code, could become the primary artifact for review." Their own open questions: will reviewers accept the spec as the thing they review, and does spec-driven review catch more bugs in practice?

The human reviewer will operate at a layer mostly above source code. How are the abstractions, specifications and design changing?

What a change review would look at

1. **The project's own abstractions and specifications.** State machines, sequence diagrams, APIs and schemas, correctness properties. Linked to the code, so a change that affects them includes changes to them, and reviewing those changes is part of the change review. Whether the code still conforms is a check an agent can run
2. **A concise description of the change and its intent** beyond what those specifications capture: what AfterVibe is reaching for, written from the session and the diff
3. **Keep the per-change descriptions in the project's history.** A record of intent that people and agents can read when asking why the code is the way it is: Storey's intent debt, paid down one change at a time

The reviewer judges the design and the intent. The diff is there to drill into, not to read.

What skills should CS students have when they graduate?



From programmers to engineers

Ciera Jaspan, Engineering Productivity Research, Google, May 2026:

> "I kind of like that this is maybe going to be the forcing function that's going to take us from being programmers to being engineers. The code is still there. We still have to make sure it's high quality. But part of quality is not just the individual lines of code. It's about how the whole system is put together. It's about, does this system even solve the problem my users have in the first place?"

And earlier in the same session:

> "we're now going to need to make sure everybody is a senior engineer, including our most junior people graduating from college today."

What the people who hire say

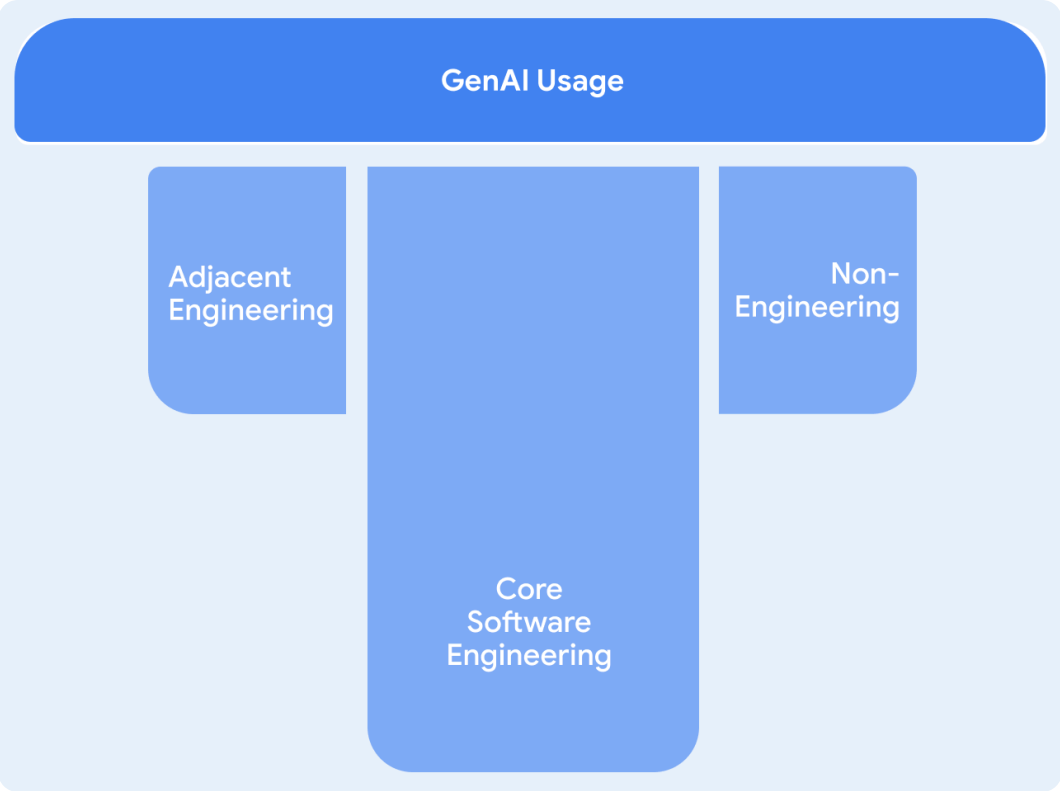


Figure 3, Kam et al., FSE 2025 (CC BY 4.0)

Kam et al., Google + Education Development Center, FSE 2025 (job analysis with expert developers, May 2024, before agents)

Deep in **core software engineering**. Broad in using AI, in adjacent engineering (security, regulation), and in the **non-engineering** context: users, customers, business, market.

"a regular AI-enhanced developer of tomorrow will need to have the skills & knowledge of today's senior developer"

Skills that used to be learned late are becoming entry requirements

Specification. Testing and quality assurance. Version control. Design. Reading and explaining code you did not write. Working with people who are not software developers.

Students used to pick these up in senior courses, internships, side projects, or the first year on the job.

We could have been teaching many of these fifteen years ago. Some CS programs were.

Depth and *detectors*

Shriram Krishnamurthi, "Advice to the New College Student in the AI Era," summer 2026:

> "the purpose of going to college is to emerge as close to middle-aged as possible … cognitively. The way you do that is to acquire depth."

> "the scars of battle help you develop a finely-tuned sense of what can go wrong. This translates into high-quality detectors that identify various kinds of weaknesses: bullshit, threats to validity, ground not covered, and so on."

Detectors = a person's internal alarms. Not a tool.

> "Long, manual slogs helped us build up those detectors, and now we have tools that amplify the need for them. Without those detectors, modern AI tools set you up for epic failure — and the worst part is, without the detectors, you won't even know you've failed."

Software is more than source code and unit tests

Software includes decisions and thinking about:

1. **What** the software must do, and what properties it must have
2. **How** it accomplishes that: strategies, algorithms, architecture, interfaces
3. **Gaining confidence** that what was built serves its intended purpose

These concerns are spread across many documents and artifacts: requirements, design documents, diagrams, API and schema definitions, tests, conventions files, review discussions, and the code itself. Often jumbled.

Confidence is bigger than tests: exposure notification

At Apple, on COVID exposure notification:

- Nothing logged in the clear on Apple servers. Not even the number of users in a region
- Opt-in privacy-preserving analytics: you could only learn what you had decided **in advance** to ask, and devices added noise
- We needed two significant revisions. We had missed the most important question: **are people who get a notification more likely to contract COVID?**
- After vaccines, base rates fell and the noise swamped the signal. We had to argue for new analytics
- The analytics also found bugs in Apple's and Google's code that we would never have known about

Design documentation is where humans matter most right now

Sequence diagrams. State machines. Externally exposed APIs you cannot refactor after release. Complex types and schemas.

Some might have just lived in developers' heads, or in a document somewhere that did not track revisions with the code and quietly went stale. Now: **humans author and review the design; models keep it in sync with the code and tests.**

> "the responsibilities of software developers are rapidly becoming more about designing and less about programming." Shaw, Hilton, Fairbanks, *IEEE Software*, Mar-Apr 2026

Caution (Storey): "Resist the automation of understanding."

What do we know about how AI affects learning?



billpugh.github.io/talks/2026-10-12-UCSD

What counts as learning?

- Learning is not the same as getting the task done or the score on the test
- Acquiring a skill requires mental effort and, usually, productive struggle and failure along the way
- The research question is not "do students perform better with AI" but "what skills have students acquired"
- Many published papers on the impact of AI on student learning date from before agentic AI coding tools were good. Impact of current and future models uncertain

Performance is not learning

Bastani et al., PNAS 2025 (about 1,000 high-school math students, randomized):

- With unguarded GPT-4 during practice: **+48%** on practice problems
- On the unassisted exam: **–17%** versus students who never had access
- With a tutor scaffold that withheld answers: harm removed, no gain

Anthropic, Jan 2026 (52 junior engineers learning a Python library): AI-assisted group **50%** on the follow-up quiz vs **67%** hand-coding. The AI group finished only two minutes faster.

You can cheat yourself by completing the task without obtaining the learning the task was designed to provide, even if you are allowed to use AI.

Not specific to CS: a randomized trial across UMD

Liu, Sweet, Chen, et al., University of Maryland, preprint dated Sept 29, 2026. Fall 2025: 30 instructors in 11 of UMD's 12 colleges, 2,379 students. Instructors were randomized to turn on UMD's course-grounded GPT-4o study assistant.

- Only **15%** of the students offered it ever used it
- Among sections of the same course: final grades **0.37 SD lower** (about 4 points of 100); activity on the course site **0.9 SD lower**
- Students who never opened it lost about as much as those who did. Larger losses for first-generation students
- **74%** of requests asked for an answer or explanation. **0.7%** asked for feedback on the student's own work

Putting an AI tool in a course is not the same as integrating it into the teaching. It was left in answer-giving mode, and no instructor built assignments around it.

Struggle has to land on the right thing

- Bjork: a difficulty is desirable only if it exercises the process you are trying to build. A logic error in a loop exercises how programs run. A missing colon does not
- Kapur, "productive failure": students who first attempt problems they have not yet been taught how to solve, and then get instruction, learn more than students who get the instruction first and then practice, **provided the failure is safe and is followed by consolidation.** A meta-analysis of 53 studies puts the effect at about a third of a standard deviation, larger when done well
- Krishnamurthi's test for the student: **"Am I eliminating failure?"**

Students who skip the learning strategies do not find out until it is too late

- Prather et al. 2024 (21 CS1 students, think-aloud with Copilot and ChatGPT): new metacognitive difficulties when coding with AI: being interrupted by suggestions, being misled down the wrong path, and being behind without knowing it
- Denny et al. 2025, 1,028 intro students at the University of Auckland working on underspecified problems: about **95%** of A students probed the oracle before coding; **more than half** of D students never did
- Bastani's students who practiced with AI felt fine right up to the exam

If the course only *recommends* the strategies that lead to learning, the students who most need them will not adopt them. The course has to push everyone into them.

Adding AI to a course shouldn't make it easier; maybe harder

For both students and instructor.

- Need to decide what the important concepts and skills this course builds
- It can't be just "complete a task"
- Deftly navigate the use of AI so students productively struggle on tasks that build those important skills, while minimizing the struggle with unimportant ones

Two obvious ways to patch an existing programming course

1. **Ban AI** on programming assignments
2. **Allow AI at the student's discretion**, but leave the content unchanged from before AI, and what students are responsible for on exams unchanged

A coherent case can be made for 1, and some courses may do it until we have better approaches. But it cannot be enforced.

2 could end badly. Some students will avoid the challenges needed to learn skills that are still being tested, and will not find out until the exam. Unfortunately, 2 is what many courses have drifted into.

Anything beyond patching means moving topics around and re-imagining the course.

What advice can we give current CS students?



Take a course in modern software development

If your CS department offers one, take it.

Specification, testing, review, design: these are the skills that are becoming entry requirements, and they change far more slowly than the tools.

Decide what you want to master, and what you don't

Productively struggle with the concepts and skills you want to know well.

You might decide you never want to master SQL syntax.

Maybe you decide you don't want to master building the front end for a web app. Then you can delegate most of that to AI, although you are still responsible for thinking about the entire application and the messages that will be exchanged with the front end.

Don't race to keep up with the bleeding edge of AI coding tools

It is exhausting, and whatever you learn is out of date in six months.

You do have to experience it for yourself, once, properly. Jaspan learns one new tool a month: "It keeps me sane." That is a full-time Google engineer's pace. With a full course load, something new about three times a year, learned properly, is enough.

Have something software-related going on outside your courses

With people who are not all CS students.

Don't depend on the CS courses offered in the next two years to be the only thing you need to get you there. Find a group helping some organization solve a need in which software is a key component: a club such as Hack4Impact, a startup idea, a field outside CS you come to understand deeply.

I've observed over decades as a student and a faculty member that the students best prepared to have impact all had something like this. Sometimes research with a faculty member. More often, something built with other students. True for decades. More important now.

What that builds: taste

Calibrated judgment on the dimensions nobody has written a test for:
what is missing, what would be simpler, what will be hard to change, what the user actually wanted.

- Scarce because it cannot be measured (Google's 117 metrics found none that worked) and because the places it was learned, code review and asking a senior, are eroding
- Built the way every design discipline builds it: many examples with feedback, and comparison. The studio critique
- Agents make comparison cheap: five designs in a minute

How should the CS curriculum change, from the first programming course to the capstone?

billpugh.github.io/talks/2026-10-12-UCSD

Changing a curriculum

- A CS department cannot reinvent its courses every year or two
- A course has to reuse **60 to 90% of its material** from one year to the next, or it cannot be staffed, approved, or taught well
- So the question is not "what should the course look like this fall" but "what can we build that stays mostly right for several years"
- Fundamentals change over decades. Effective use of AI tools changes in months. Design the course so the perishable part is small and swappable

Every layer changes

Layer	What changes
Intro	Concepts without the syntax tax; specification; verification
Data structures, systems	Read and audit generated code; invariants; performance, memory, behavior under stress
Software engineering	Design documentation, API design, types and schemas, review as norm-setting, the three debts
Capstone	An intent-debt laboratory

Experiments are wanted throughout. **The intro course is where they are urgently needed.**

How should we change our intro CS courses?



Why the intro courses are the hard case

- The typical assignments are so easy that even non-frontier models produce solutions to all of them
- The problems are too simple and unrealistic to engage students
- And yet these courses may be where students acquire the fundamental conceptual building blocks: iteration, lists, conditionals, functions, and how programs run
- Without that conceptual foundation, students might struggle to become good software developers, even in a world where they never type source code

Additional complication: many students will skip CS1

- We want our new CS1 to be approachable by students with no previous programming experience
- Many students will likely skip CS1, based on work in high school or community college, in courses that have not been updated for generative AI
- Any topic we consider required for using generative AI effectively has to be covered in CS2, the first CS course some students will take

For consideration: an experimental CS1 course

Students learn the **core concepts of programming** while spending a **minimum of time, friction, and struggle on source code syntax.**

- Stick to basic constructs that look much the same in many languages: loops, conditionals, arrays, maps, sets, functions, recursion
- Let students see the same code in Java, Python, C, and Swift, and notice what each language makes you decide
- Program reading and tracing. Faded Parsons problems. Writing tests that distinguish 4 correct implementations from 8 incorrect ones. Probeable problems
- Struggle on the right thing

The Rainfall problem

> Write a function that takes a list of daily rainfall amounts and returns the average rainfall, or `None` if the average can't be determined. Ignore negative values.

Rainfall: what forty years of studies found

For those of us who have written code for decades it is hard to see how a student struggles with this. Some do.

- Soloway, Yale, 1983: **14%** of CS1 students solved it
- Forty years of studies: the failures are the empty-input and negative-value corner cases, not the loop
- Any model solves it now. In 2023 Copilot failed it the way students do: it forgot the empty list

The source code that solves this is not interesting. **Thinking about the corner cases is the skill students need to develop.**

> "Most bugs won't be 'coding' bugs. They'll be 'you asked for the wrong thing' bugs." Thorsten Ball, Sept 2026

Rainfall as a Parsons problem

The lines of a working program, scrambled. The student puts them in order and indents them. Nothing to type, no syntax errors.

> Write a function that takes a list of daily rainfall amounts and returns the average rainfall, or `None` if the average can't be determined. Ignore negative values.

Where two blocks sit side by side, only one belongs.

```
def average_rainfall(amounts):
return total / count
if amount > 0:
```
or
```
if amount >= 0:
```
```
count = count + 1
for amount in amounts:
return None
total = total + amount
if total == 0:
```
or
```
if count == 0:
```
```
total = 0
count = 0
```

Rainfall as a Parsons problem: the answer

```
def average_rainfall(amounts):
    total = 0
    count = 0
    for amount in amounts:
        if amount >= 0:
            total = total + amount
            count = count + 1
    if count == 0:
        return None
    return total / count
```

- `>= 0`, not `> 0`: a day with no rain is still a day
- `total` and `count` change together, under the same `if`: ignored values stay out of the denominator
- `if count == 0`, not `if total == 0`: two dry days average 0.0; only no days at all is "no data"
- `None` says "no data"; 0 would say "no rain"

The student typed nothing, and still had to decide every corner case.

Faded Parsons problems

Real code, visible. Lines to arrange. **Blanks for the variables the student must name, or for whole expressions.** Paired distractors can still encode the classic bugs (`if ___ > 0` next to `if ___ >= 0`).

- Deciding what variables you need and what they each mean is an important skill
- Weinman, Fox, Hearst (Berkeley, 2021): transfer to code writing at least as good as writing, in about a third less time, preferred three to one
- Never run as the primary format for a whole semester. That is part of the experiment
- A language toggle: Python by default; later, the same problem in Java, C, Swift, or pseudocode, with one question: what did this language make you decide?

The verification unit

How do you know the answer is right when you did not compute it?

- Properties, not expected values
- Equivalent answers, not equal answers: a simple equality test on outputs fails when several different outputs are all correct
- A slow, obviously correct second implementation, and compare them
- Invariants checked after every operation
- Metamorphic relations: `avg(2·xs) = 2·avg(xs)`
- Coverage is a floor. Mutation testing: test the tests
- **An exercise:** 12 implementations, pairwise distinguishable; 4 correct, 8 wrong. Write tests that tell them apart

The one principle: the oracle must be **independent** of the implementation. Tests written by LLMs after seeing the code catch 14% of faults instead of 25%.

Ambiguity, on purpose

Probeable Problems (Denny et al., 2025): the spec omits details. "Find the smallest even value." An oracle answers any input you try. Probes are free; failed submissions cost.

Top students probe first. More than half of the weakest never do.

Brown's version: students write a test plan before coding; the rows they cannot fill in without a decision are the ambiguities, and the plan says **"decide"** there.

When do we cover agentic AI?

- Honestly, I don't know yet. Where agentic AI belongs in CS1 needs a much more detailed course design before anyone can say
- Perhaps this is best covered more in CS2, which fewer students will skip
- Perhaps allow students to use LLMs to generate code to parse input or display/graph output
- Under the covers, the course will likely use agentic AI for generating and analyzing Parsons problems, and more

Questions?

Backup slides



Backup: Parsons problems, the evidence

- Ericson, Margulieux, Rick 2017: Parsons 473 s vs fix 679 s vs write 714 s per session; no difference in learning or one-week retention
- Denny, Luxton-Reilly, Simon 2008: Parsons marks correlate with writing marks, not tracing; lower-quartile scores: tracing 0%, writing 48%, Parsons 78%
- Weinman 2021: faded Parsons beat writing on pattern acquisition; on code-writing transfer, 85% vs 75% in one study, 28% vs 29% in the other
- 13-institution study 2023: Parsons practice erased the prior-familiarity advantage on the final code-writing task
- Limits: near transfer only; expertise reversal for experienced students; GPT-4V solved 96.7% of Parsons screenshots, so not an assessment format

Backup: the verification techniques, with references

- Coverage is a floor: Inozemtseva & Holmes, ICSE 2014
- Mutation testing: Jia & Harman, TSE 2011; Just et al., FSE 2014; Code Defenders (Fraser et al.)
- Property-based testing: Claessen & Hughes, QuickCheck, ICFP 2000; Hypothesis; Wrenn, Nelson, Krishnamurthi 2021 on relational problems
- Differential testing: McKeeman 1998; caveat Knight & Leveson 1986
- Metamorphic testing: Chen et al., ACM CSUR 2018
- The oracle problem: Barr et al., TSE 2015; Weyuker 1982
- Examplar: Wrenn & Krishnamurthi, ICER 2019; conceptual mutation testing: Prasad et al. 2024

Backup: Kam et al., method and the T

- DACUM job analysis: 8 expert developers (screened from 3,000), 13 Google advisors, 2.5-day workshop May 31 to June 2, 2024; 12 goals, 75 tasks
- Six-step task workflow: Identify, Engage, Evaluate, Calibrate, Tweak, Finalize. "The use of GenAI tools is perhaps often understood to entail prompting an LLM only. Unfortunately, this corresponds to only a subset of the Engage step"
- The profile has a seventh step the paper dropped: Socialize. Getting the artifact accepted by the team
- Limits: 8 people, 7 men, self-report, pre-agent

Backup: industry numbers and their caveats

- Cherny, June 2026: "I haven't written a line of code by hand in, I think, eight months now"; Anthropic spokesperson, Jan 2026: 70 to 90% company-wide
- METR RCT (early-2025 tools): experienced developers **19% slower** with AI; they predicted +24% and believed afterward +20%. METR has since said its design broke because developers refused to work without the tools
- LinearB (8.1M PRs, 2025): AI PRs wait 4.6x longer for first review; accepted 32.7% vs 84.4%
- DORA 2025: 90% use AI daily; 30% trust its output little or not at all; throughput up, stability down
- Stack Overflow 2025: 84% use or plan to use AI; 29% somewhat trust it, 3% highly
- GitClear (623M changed lines, vendor): duplication +81%, refactored code 21% to 3.8%, edits to year-old code −74%

Backup: design rules versus tests (Yu et al., 2026)

- 1,787 design rules mined from the code-review threads of six real projects (Django, SymPy, Ansible, Teleport, Flipt, Open Library); 495 issues
- Best agent: passed 77% of tests, fully satisfied the design rules on 35%; 42% of its patches passed while violating a rule
- Passing the tests predicted design compliance about as well as a coin flip (Cramér's V at most 0.12)
- Handing agents the relevant rules cut violations by about six points; pass rates mostly got worse
- Example: Django #13410, the rule was catch BlockingIOError only; the agent caught OSError

Backup: Cornell, September 2025

Students (2,029): 56% use GenAI several times a week or daily; 68% never use it to complete entire assignments, 7% often; 58% say it will be essential to their future work.

Faculty (737): 61% worried students gain an unfair advantage; 57% say it has challenged their ability to assess learning; 28% encourage students to use it; 16% think it has improved teaching and learning.